192325039

Experiment-1

Aim:- To write a python program to solve the 8-puzzle problem using the A* search algorithm.

Algorithm:-

- Start with the initial state and define the goal state.
- Calculate the Manhattan distance heuristic for each state.
- Select the state with the lowest evaluation value.
- Find the blank space (0) in the puzzle.
- Generate new states by moving the blank space up, down, left, or right.
- Repeat until the goal state is reached.
- Display the solution path.

Program:-

```
from heapq import heappush,
heapop
goal = (1,2,3,4,5,6,7,8,0)
def heuristic(state):
  distance = 0
  for i, value in enumerate(state):
    if value != 0:
      goal_pos = value - 1
      distance += abs(i//3 - goal_pos//3)
      return distance
  def solve(start):
    PQ = []
    heappush(PQ, (heuristic(start), 0, start, []))
```

```
    visited = set()

    while pq:
        f, g, state, path = heappop(pq)
        if state in visited:
            continue
        visited.add(state)

        if state == goal:
            for step in path + [state]:
                print(step[:3])
                print(step[3:6])
                print(step[6:])
                print()
            return
        zero = state.index(0)
        row, col = zero // 3, zero % 3
        for dr, dc in [(0,1),(0,-1),(1,0),(-1,0)]:
            nr, nc = row+dr, col+dc
            if 0 <= nr < 3 and 0 <= nc < 3:
                new_pos = nr*3 + nc
                new_state = list(state)
                new_state[zero],
                new_state[new_pos] = (new_state[new_pos],
                new_state[zero])
                new_state = tuple(new_state)
                if new_state not in visited:
                    heappush(pq, (g+1+heuristic(new_state),
                                  g+1, new_state, path+[state]))
        print("No solution found")
    start = (1,2,3,
             4,0,6,
             7,5,8)
    solve(start)
```

output:-

(1,2,3)
(4,0,6)
(7,5,8)

(1,2,3)
(4,5,6)
(7,0,8)

(1,2,3)
(4,5,6)
(7,8,0)

Experiment-2

Aim:- To write a Python program to place eight queens on a chessboard so that no two queens attack each other.

Algorithm:-

- Initialize an 8x8 chessboard.
- Start placing a queen in the first row.
- Check whether the selected position is safe.
- If safe, place the queen and move to the next row.
- If no safe position is available, backtrack to the previous row.
- Repeat until all eight queens are placed.
- Display the chessboard.

Program:-

```
N=8
def print_board(board):
    for row in range(N):
        for col in range(N):
            if board[row]==col:
                print("Q",end=" ")
            else:
                print(".",end=" ")
        print()
def is_safe(board,row,col):
    for i in range(row):
        if board[i]==col:
            return False.
```

```
        if abs(board[i]-col)==abs(i-row):
            return False
    return True

def solve(board, row):
    if row==N:
        print_board(board)
        return True

    for col in range(N):
        if is_safe(board,row,col):
            board[row]=col

            if solve(board,row+1):
                return True

    return False
board=[-1]*N

if not solve(board,0):
    print("Solution does not exist")
```

Output:-

```
Q.......
...Q....
.......Q
....Q...
..Q.....
.....Q..
.Q......
...Q....
```

192325039

Experiment-3

Aim:- To write a python program to solve the water jug problem using Breadth-First Search.

Algorithm:-

- Initialize two jugs with capacities of 4 Lit and 3 lit
- Set the initial state as (0,0).
- Create a queue to store possible states.
- Generate states by filing, emptying, and transferring water between the jugs.
- Check whether either jug contains the target quantity of 2 litres.
- Repeat until the target is reached or no solution exists.
- Display the explored states and the result.

Program:-

```
from collections import deque
def water_jug(cap1, cap2, target): queue = deque([(0,0)])
 visited = set()
 while queue:
    x,y = queue.popleft()
 if (x,y) in visited:
    continue
 visited.add((x,y))
 print(x,y)
 if x == target or y == target:
      print("Target reached")
```

```
            return
        states = [(cap1,y), (x,cap2), (0,y), (x,0), (x-min(x, cap2-y),
                  y+min(x,cap2-y)), (x+min(y, cap1-x), y-min(
                  y, cap1-x))]
        for state in states:
            if state not in visited:
                queue.append(state)
        print("Solution not possible")
water_jug(4,3,2)
```

Output:-

```
0 0
4 0
0 3
4 3
1 3
3 1
1 0
3 3
4 2
Target reached.
```

192325039

Experiment-4

Aim:- To write a python program to solve a crypt-arithmetic problem by aligning unique digits to different letters.

Algorithm:-

1. Define the words SEND, MORE, and MONEY.
2. Identify all unique letters in the words.
3. Generate possible digit alignments using permutations.
4. Ensure that each letter is aligned a unique digit.
5. Check that the first letters of the words are not zero.
6. Verify whether SEND+MORE=MONEY.
7. Display the solution when a valid alignment is found.

Program:-

```
from itertools import permutations
letters = "SENDMORY"
for digits in permutations(range(10), len(letters)):
    d = dict(zip(letters, digit))
    if d["S"] == 0 or d["M"] == 0:
        continue.
    send = (d["S"]*1000+d["E"]* 100 +d["S"]*10+d[D]
    more = ( d["M"]*1000+d["O"]* 100 + d["R"]*10+d["E"]
    money = (d["M"]*10000+ d["O"]*1000+d["N"]*100+d["E"]*10
            +d["Y"])
```

```
if Send+more == money
    Print("SEND=", send)
    Print("MORE=", more)
    Print("MONEY=", money)
    break
```

Output:-

SEND = 9567

MORE = 1085

MONEY = 10652

Experiment-5

Aim:- To write a python program to solve the missionaries and cannibals problem using Breadth-First Search (BFS).

Algorithm:-

1. Initialize three missionaries and three cannibals on the left bank.
2. Place the boat on the left bank.
3. Allow the boat to carry one or two people at a time.
4. Generate possible states by moving people across the river.
5. Ensure that cannibals never outnumber missionaries on either bank when missionaries are present.
6. Use BFS to find a valid sequence of moves.
7. Display the solution path until everyone reaches the right bank.

Program:-

```
from collections import deque
def is_valid(m,c):
    if m<0 or c<0 or m>3 or c>3:
        return False.

    if m>0 and m<c:
        return false.
```

```
        mr = 3 - m
        cr = 3 - c
    if m > 0 and mr < cr:
        return False
    return True

def Solve():
    start = (3, 3, 0)
    goal = (0, 0, 1)

    queue = deque([[start, [start]]])
    visited = {start}

    moves = [
        (1, 0),
        (2, 0),
        (0, 1),
        (0, 2),
        (1, 1),
    ]
while queue:
    state, path = queue.popleft()
    if state == goal:
        for m, c, boat in path:
            side = "Right" if boat == 1
else "Left"
            print("Missionaries:", m, "Cannibals:", c,
                  "Boat:", side)
            return
        m, c, boat = state

    for dm, dc in moves:
            if boat == 0:
                new_state = (m - dm, c - dc, 1)
            else:
```

```
        new_state = (m+dm, c+dc, 0)
        if (is_valid(new_state[0], new_state[1])
            and new_state not in visited):
            visited.add(new_state)
            queue.append((new_state, path +
                [new_state]))
        print("No solution found")
solve()
```

Output:-

```
Missionaries: 3 Cannibals: 3 Boat: Left
Missionaries: 3 Cannibals: 1 Boat: Right
Missionaries: 3 Cannibals: 2 Boat: Left
Missionaries: 3 Cannibals: 0 Boat: Right
Missionaries: 3 Cannibals: 1 Boat: Left
Missionaries: 1 Cannibals: 1 Boat: Right
Missionaries: 2 Cannibals: 2 Boat: Left
Missionaries: 0 Cannibals: 2 Boat: Right
Missionaries: 0 Cannibals: 3 Boat: Left
Missionaries: 0 Cannibals: 1 Boat: Right
Missionaries: 0 Cannibals: 2 Boat: Left
Missionaries: 0 Cannibals: 0 Boat: Right
```